

CTF – Crack the Hash

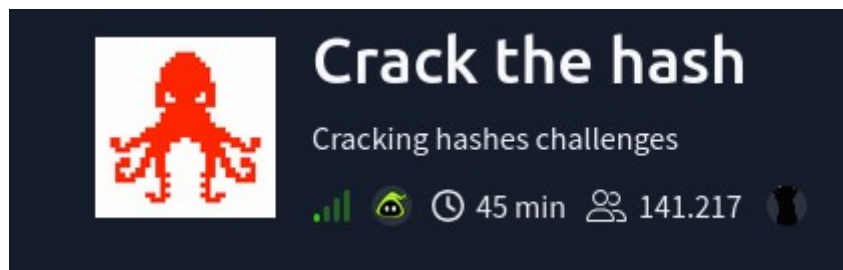
"Crack the hash" (quebrar o hash) é o processo de reverter um código hash — uma sequência alfanumérica de tamanho fixo gerada a partir de dados (como senhas) — para encontrar o seu valor original. Como funções hash são unidirecionais, o "crack" envolve técnicas como força bruta ou dicionários para adivinhar a senha que gerou aquele hash.

Ferramentas utilizadas:

Kali Linux

hash-identifier

hashcat



LISTA 1

A primeira hash a ser quebrada é essa
48bb6e862e54f2a795ffc4e541caed4d. Utilizando o comando hash-identifier:

```
(kali㉿kali)-[~]
$ hash-identifier

#####
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#                                     #
#####

HASH: 48bb6e862e54f2a795ffc4e541caed4d

Possible Hashs:
[+] MD5
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))
```

O hash-identifier nos retorna os algoritmos com maior probabilidade de correspondência sob a seção 'Possible Hashes'.

com maior probabilidade de quebrar essa hash é o **MD5**. Utilizando o comando **'hashcat -a 0 -m 0 48bb6e862e54f2a795ffc4e541caed4d /usr/share/wordlists'**, a primeira flag é descoberta: 'easy'

A seguinte hash a ser quebrada é 'CBFDAC6008F9CAB4083784CBD1874F76618D2A97', que utilizando o mesmo procedimento, descobrimos que o algoritmo com maior probabilidade dessa vez é o SHA-1, que possui o código 100 no hashcat.

Os códigos das hashes podem ser conferidas no site
https://hashcat.net/wiki/doku.php?id=example_hashes

O comando a ser utilizado para quebrar a hash é **'hashcat -a 0 -m 0 CBFDAC6008F9CAB4083784CBD1874F76618D2A97 /usr/share/wordlists'** que retorna 'password123' como resposta.

A próxima hash a ser quebrada é 1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032, utilizando novamente o hashid, o algoritmo da vez é o SHA-256 de código 1400.

Inserindo **'hashcat -a 0 -m 1400**

1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032 /usr/share/wordlists' temos 'letmein' como flag.

Embora a ferramenta hash-identifier não tenha reconhecido automaticamente o algoritmo da hash

\$2y\$12\$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom, foi possível determinar sua natureza por meio da análise de padrões em bases de exemplos. O prefixo 2y identifica o algoritmo bcrypt (Blowfish), que no Hashcat é operado sob o código de modo 3200. Para otimizar a quebra, considerando que a senha possivelmente segue um padrão curto, utilizou-se o comando grep para filtrar a wordlist **rockyou.txt**. O objetivo foi extrair apenas termos compostos por quatro letras minúsculas, gerando um dicionário personalizado:

grep -E '^[a-z]{4}\$' /usr/share/wordlists/rockyou.txt > list.txt

Com a lista refinada, executou-se o ataque de dicionário através do seguinte comando:

hashcat -a 0 -m 3200 hash.txt list.txt

A execução resultou na identificação bem-sucedida da senha: **"bleh"**.

No último desafio deste nível, o hash-identifier não apontou o MD4 como uma das principais possibilidades, apesar de a dica do exercício confirmar que este era o algoritmo correto.

Durante as tentativas de quebra com o Hashcat, o processo resultava persistentemente no status Exhausted, indicando que a senha não constava na wordlist utilizada. Diante disso, recorri a um decriptador de MD4 online para processar a hash, o qual retornou com sucesso a credencial: **"Eternity22"**.

Task 1 Level 1

Can you complete the level 1 tasks by cracking the hashes?

Answer the questions below

48bb6e862e54f2a795ffc4e541caed4d

easy

✓ Correct Answer

?

CBFDAC6008F9CAB4083784CBD1874F76618D2A97

password123

✓ Correct Answer

?

1C8BFE8F801D79745C4631D09FFF36C82AA37FC4CCE4FC946683D7B336B63032

letmein

✓ Correct Answer

?

\$2y\$12\$Dwt1BZj6pcyc3Dy1FWZ5ieeUznr71EeNkJkUlypTsgbX1H68wsRom

bleh

✓ Correct Answer

?

279412f945939ba78ce0758d3fd83daa

Eternity22

✓ Correct Answer

?



Lista 2:

A ferramenta hash-identifier classificou a hash F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85 como sendo do tipo **SHA-256**, um algoritmo já abordado em etapas anteriores.

Para realizar a quebra, utilizou-se o Hashcat com o modo correspondente (-m 1400) e a wordlist padrão *rockyou.txt*:

```
hashcat -a 0 -m 1400
```

```
F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85 /usr/share/wordlists/rockyou.txt
```

A execução do comando retornou a credencial: “**paule**”.

A hash seguinte apresentou a mesma inconsistência de identificação: embora o **hash-identifier** tenha sugerido o algoritmo **MD5**, a dica do desafio confirmava tratar-se de uma hash **NTLM**. No **Hashcat**, esse tipo de criptografia é processado sob o código de modo **1000**.

Ciente dessa divergência, executei o ataque de dicionário com a *rockyou.txt*:

```
hashcat -a 0 -m 1000 1DFECA0C002AE40B8619ECF94819CC1B  
/usr/share/wordlists/rockyou.txt
```

O procedimento foi finalizado com sucesso, revelando a flag:
“**n63umy8lkf4i**”.

Como o **hash-identifier** falhou novamente em reconhecer o tipo de hash, fiz uma busca manual baseada no prefixo. Identifiquei que hashes começando com **6** pertencem ao algoritmo **sha512crypt (SHA512 Unix)**, que utiliza o código **1800** no Hashcat. Um detalhe importante é que esse formato já inclui o *salt* dentro da própria hash, então não precisamos nos preocupar com isso separadamente.

Para realizar a quebra de forma eficiente, segui estes passos:

Criação do arquivo: Salvei a hash em um arquivo .txt, já que o terminal pode causar erros ao interpretar os caracteres especiais do SHA-512 diretamente.

Filtragem da Wordlist: Usei o comando **grep** novamente para selecionar apenas palavras com exatamente seis caracteres na *rockyou.txt*.

```
grep -E '^.{6}$' /usr/share/wordlists/rockyou.txt > list.txt
```

Por fim, rodei o comando de ataque:

```
hashcat -a 0 -m 1800 hash.txt list.txt
```

Resultado: A senha encontrada foi "**waka99**".

Para o último desafio, o **hash-identifier** apontou o algoritmo **SHA-1**. Contudo, como a hash dependia de um *salt* específico fornecido pelo

desafio (tryhackme), identificou-se que o modo correto de operação seria o **HMAC-SHA1**, referenciado pelo código **160** no Hashcat.

```
e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme:481616481616  
Session.....: hashcat  
Status.....: Cracked  
Hash.Mode.....: 160 (HMAC-SHA1 (key = $salt))  
Hash.Target.....: e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme
```

Dessa forma, a execução foi realizada combinando a hash e o *salt* no formato apropriado, utilizando a wordlist *rockyou.txt*:

```
hashcat -a 0 -m 160
```

```
e5d8870e5bdd26602cab8dbe07a942c8669e56d6:tryhackme
```

```
/usr/share/wordlists/rockyou.txt
```

O procedimento revelou a última resposta necessária para concluir o CTF: **“481616481616”**.

Task 2 **Level 2**

This task increases the difficulty. All of the answers will be in the classic [rock you](#) password list.

You might have to start using hashcat here and not online tools. It might also be handy to look at some example hashes on [hashcats page](#).

Answer the questions below

Hash: F09EDCB1FCEFC6DFB23DC3505A882655FF77375ED8AA2D1C13F640FCCC2D0C85

paule

✓ Correct Answer

Hash: 1DFECA0C002AE40B8619ECF94819CC1B

n63umy8lkf4i

✓ Correct Answer

?

Hash: \$6\$aReallyHardSalt\$6WKUTqzq.UQqmrmp/T7MPpMbGNnzXPMAXi4bJMI9be.cf13/qxlf.hsGpS41BqMhSrHVXgMpdjS6xeKZAs02.

Salt: aReallyHardSalt

waka99

✓ Correct Answer

Hash: e5d8870e5bdd26602cab8dbe07a942c8669e56d6

Salt: tryhackme

481616481616

✓ Correct Answer

?

